

Day 4：数据库技术应用 *□

*□今日时间安排（半天强化版）

时段	时间	内容	形式
上午	9:00-10:30	PostgreSQL高级特性与优化	理论+实践
	10:45-12:00	Redis数据结构与缓存策略	动手实验
下午	14:00-15:30	数据库设计与ORM进阶	项目实战
	15:45-17:00	缓存与会话管理实战	完整应用

✓✓ 学习目标

□今日完成后，你将能够：

✓ 设计高效数据库 - 规范化设计、索引优化、查询性能调优 ✓ 运用PostgreSQL特性 - JSON支持、全文搜索、窗口函数、CTE ✓ 掌握Redis数据结构 - String、Hash、List、Set、Sorted Set ✓ 实现缓存策略 - 缓存穿透/击穿/雪崩解决方案 ✓ 管理用户会话 - Redis Session、JWT Token存储 ✓ 数据库监控 - 慢查询分析、连接池配置

** 详细学习内容

□1. PostgreSQL 高级特性（1.5小时）

□1.1 数据库设计原则

三大范式：

```
-- 第一范式 (1NF): 列不可再分
-- □错误示例
CREATE TABLE bad_design (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    contacts TEXT -- 存储多个电话号码, 用逗号分隔
);

-- □正确示例: 拆分为关联表
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE user_contacts (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    phone VARCHAR(20) NOT NULL,
    type VARCHAR(20) CHECK (type IN ('mobile', 'home', 'work'))
);

-- 第二范式 (2NF): 非主属性完全依赖于主键
-- 第三范式 (3NF): 非主属性不传递依赖于主键
```

反规范化权衡:

```

-- 场景：电商订单系统需要频繁显示商品名称
-- 方案A：严格规范化（多表JOIN）
SELECT
    o.order_id,
    o.order_date,
    p.product_name,      -- 从products表JOIN
    oi.quantity,
    oi.price
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id;

-- 方案B：反规范化（冗余存储商品名，减少JOIN）
CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(id),
    product_id INTEGER REFERENCES products(id),
    product_name VARCHAR(200),  -- 冗余字段
    quantity INTEGER NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

-- 选择建议：
-- 读多写少 * 可适当反规范化
-- 写入频繁 * 保持规范化
-- 需要实时一致性 * 必须规范化

```

□1.2 高级SQL特性

公用表表达式（CTE）：

-- 基础CTE: 提高可读性

```
WITH monthly_sales AS (  
    SELECT  
        DATE_TRUNC('month', created_at) as month,  
        SUM(amount) as total_sales,  
        COUNT(*) as order_count  
    FROM orders  
    WHERE status = 'completed'  
        AND created_at >= '2024-01-01'  
    GROUP BY DATE_TRUNC('month', created_at)  
)  
SELECT  
    month,  
    total_sales,  
    order_count,  
    LAG(total_sales) OVER (ORDER BY month) as prev_month_sales,  
    ROUND(  
        (total_sales - LAG(total_sales) OVER (ORDER BY month)) /  
        LAG(total_sales) OVER (ORDER BY month) * 100,  
        2  
    ) as growth_rate  
FROM monthly_sales  
ORDER BY month;
```

-- 递归CTE: 组织架构树

WITH RECURSIVE org_tree AS (

-- 基础查询: 根节点 (CEO)

```
SELECT  
    id,  
    name,  
    title,  
    manager_id,  
    1 as level,  
    ARRAY[id] as path  
FROM employees  
WHERE manager_id IS NULL
```

UNION ALL

-- 递归查询: 下属员工

```
SELECT  
    e.id,  
    e.name,  
    e.title,  
    e.manager_id,  
    ot.level + 1,  
    ot.path || e.id  
FROM employees e
```

```
JOIN org_tree ot ON e.manager_id = ot.id
```


SELECT

level,


```
REPEAT(' ', level - 1) || name as indented_name,
```

title,

path

FROM org_tree

```
ORDER BY path;
```

窗口函数:

-- 排名函数

```
SELECT
    product_name,
    category,
    sales_amount,
    RANK() OVER (PARTITION BY category ORDER BY sales_amount DESC) as rank_dense,
    DENSE_RANK() OVER (PARTITION BY category ORDER BY sales_amount DESC) as dense_rank
,
    ROW_NUMBER() OVER (PARTITION BY category ORDER BY sales_amount DESC) as row_num
FROM product_sales
WHERE sale_date = '2024-01-31';
```

-- 聚合窗口函数

```
SELECT
    order_date,
    customer_id,
    amount,
    SUM(amount) OVER (
        PARTITION BY customer_id
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) as running_total,

    AVG(amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN 3 PRECEDING AND CURRENT ROW
    ) as moving_avg_4days
FROM daily_orders
WHERE customer_id = 12345;
```

-- 分析函数：计算同环比

```
SELECT
    DATE_TRUNC('month', order_date) as month,
    SUM(amount) as monthly_revenue,

    LAG(SUM(amount)) OVER (ORDER BY DATE_TRUNC('month', order_date)) as prev_month,

    SUM(amount) - LAG(SUM(amount)) OVER (
        ORDER BY DATE_TRUNC('month', order_date)
    ) as mom_change,

    ROUND(
        (SUM(amount) / LAG(SUM(amount)) OVER (
            ORDER BY DATE_TRUNC('month', order_date)
        ) - 1) * 100,
        2
    ) as mom_growth_pct
FROM orders
GROUP BY DATE_TRUNC('month', order_date)
```

ORDER BY month;

JSON操作:

```

-- 创建包含JSON字段的表
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    attributes JSONB DEFAULT '{}',
    created_at TIMESTAMP DEFAULT NOW()
);

-- 插入JSON数据
INSERT INTO products (name, attributes) VALUES
(
    'Smartphone X',
    '{
        "specs": {
            "screen": "6.5 inch OLED",
            "ram": "8GB",
            "storage": "256GB"
        },
        "colors": ["black", "white", "blue"],
        "price": 6999,
        "in_stock": true
    }'::jsonb
);

-- 查询JSON字段
-- 获取嵌套属性
SELECT
    name,
    attributes->>'price' as price,
    attributes->'specs'->>'screen' as screen_size,
    attributes->'colors'->>0 as first_color -- 数组索引从0开始
FROM products
WHERE id = 1;

-- JSON包含查询
-- 查找所有蓝色产品
SELECT name, attributes->>'price'
FROM products
WHERE attributes @> '{"colors": ["blue"]}';

-- 查找价格大于5000的产品
SELECT name
FROM products
WHERE (attributes->>'price')::numeric > 5000;

-- 更新JSON字段
UPDATE products
SET attributes = jsonb_set(
    attributes,

```

'{price*}',

'6499'::jsonb


```
WHERE id = 1;
```


— 向数组追加元素

UPDATE products

```
SET attributes = jsonb_insert(
```

attributes,

'{colors*}', -- 追加到数组末尾使用 '{colors, -1*' 不支持, 需用 ||

```
attributes->'colors' || '"red"'::jsonb
```



```
WHERE id = 1;
```


— JSON聚合函数

SELECT

```
json_agg(attributes) as all_products_json,
```

```
json_object_agg(name, attributes->>'price') as name_to_price
```

FROM products;

全文搜索：

```
-- 创建全文搜索索引
CREATE TABLE articles (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    body TEXT,
    search_vector TSVECTOR
);

-- 创建GIN索引加速搜索
CREATE INDEX idx_articles_search ON articles USING GIN(search_vector);

-- 使用触发器自动更新搜索向量
CREATE OR REPLACE FUNCTION articles_search_vector_update() RETURNS TRIGGER AS $$
BEGIN
    NEW.search_vector :=
        setweight(to_tsvector('chinese', COALESCE(NEW.title, '')), 'A') ||
        setweight(to_tsvector('chinese', COALESCE(NEW.body, '')), 'B');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER articles_search_vector_update
    BEFORE INSERT OR UPDATE ON articles
    FOR EACH ROW EXECUTE FUNCTION articles_search_vector_update();

-- 全文搜索查询
SELECT
    title,
    ts_rank(search_vector, query) as rank
FROM articles, to_tsquery('chinese', '人工智能 & 机器学习') query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 20;

-- 高亮匹配文本
SELECT
    title,
    ts_headline('chinese', body, query,
        'MaxWords=30, MinWords=15, StartSel=<em>, StopSel=</em>'
    ) as highlighted_body
FROM articles, to_tsquery('chinese', '深度学习') query
WHERE search_vector @@ query;
```

□ 1.3 性能优化技巧

索引优化:

```
-- 查看查询执行计划
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 12345
    AND created_at >= '2024-01-01';

-- 创建合适的索引
-- 单列索引
CREATE INDEX idx_orders_customer ON orders(customer_id);

-- 复合索引（注意列顺序）
CREATE INDEX idx_orders_customer_date ON orders(customer_id, created_at DESC);

-- 部分索引（只索引满足条件的数据）
CREATE INDEX idx_active_users_email ON users(email) WHERE is_active = true;

-- 表达式索引
CREATE INDEX idx_users_lower_email ON users(lower(email));

-- 覆盖索引（包含查询所需的所有列）
CREATE INDEX idx_order_details_covering ON order_items(order_id) INCLUDE (product_id,
quantity, price);

-- 查看索引使用情况
SELECT
    indexrelname::varchar(50) as index_name,
    idx_scan as index_scans,
    idx_tup_read as tuples_read,
    idx_tup_fetch as tuples_fetched
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
ORDER BY idx_scan DESC;

-- 删除未使用的索引
DROP INDEX IF EXISTS idx_unused_index;
```

查询优化:

```

-- 问题查询：全表扫描
EXPLAIN ANALYZE
SELECT * FROM large_table
WHERE LOWER(name) LIKE '%keyword%';

-- 优化方案1：使用pg_trgm扩展进行模糊搜索
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_large_table_name_trgm ON large_table USING gin(name gin_trgm_ops);

-- 优化方案2：使用全文搜索替代LIKE
ALTER TABLE large_table ADD COLUMN search_vector TSVECTOR;
CREATE INDEX idx_large_table_search ON large_table USING gin(search_vector);
-- （然后使用全文搜索语法）

-- 分页优化（避免OFFSET过大）
-- □低效：深分页
SELECT * FROM orders ORDER BY id LIMIT 10 OFFSET 1000000;

-- □高效：基于游标的分页
SELECT * FROM orders
WHERE id > last_seen_id -- 上次最后一条记录的ID
ORDER BY id
LIMIT 10;

-- 批量操作优化
-- □低效：逐条插入
INSERT INTO logs (message, level) VALUES ('msg1', 'info');
INSERT INTO logs (message, level) VALUES ('msg2', 'error');

-- □高效：批量插入
INSERT INTO logs (message, level) VALUES
    ('msg1', 'info'),
    ('msg2', 'error'),
    ('msg3', 'warning');

-- 或使用COPY命令导入大量数据
COPY bulk_data FROM '/path/to/data.csv' DELIMITER ',' CSV HEADER;

```

□ 2. Redis 数据结构与应用（1小时）

□ 2.1 核心数据类型


```

import redis

# 连接Redis
r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

# ===== String 类型 =====
# 应用场景：缓存、计数器、分布式锁、Session存储

# 基本操作
r.set('user:1001:name', 'Alice')
name = r.get('user:1001:name')          # 'Alice'

# 设置过期时间（秒）
r.setex('session:abc123', 3600, '{"user_id": 1001*}') # 1小时后过期

# 原子递增（计数器）
r.set('page_views:home', 0)
r.incr('page_views:home')              # 1
r.incrby('page_views:home', 10)        # 11
r.decr('page_views:home')              # 10

# SETNX（仅当key不存在时设置）- 分布式锁
lock_acquired = r.setnx('lock:resource_1', 'locked')
if lock_acquired:
    try:
        # 执行临界区代码
        pass
    finally:
        r.delete('lock:resource_1')

# MSET/MGET 批量操作
r.mset({'key1': 'val1', 'key2': 'val2'})
values = r.mget(['key1', 'key2'])      # ['val1', 'val2']

# ===== Hash 类型 =====
# 应用场景：对象存储、用户信息、购物车

# 存储对象
r.hset('user:1001', mapping={
    'name': 'Alice',
    'email': 'alice@example.com',
    'age': '28'
})

# 获取字段
name = r.hget('user:1001', 'name')     # 'Alice'
user_info = r.hgetall('user:1001')     # {'name': 'Alice', 'email': '...', 'age': '2'

```


更新单个字段

```
hset('user:1001', 'age', '29')
```


获取多个字段

```
hmget('user:1001', ['name', 'email']) # ['Alice', 'alice@example.com']
```


字段是否存在

```
hexists('user:1001', 'address')      # False
```


所有字段名

```
hkeys('user:1001')
```

```
# ['name', 'email', 'age']
```


购物车示例


```
def add_to_cart(user_id, item_id, quantity):
```

```
key = f'cart:{user_id*}'
```

```
r.hincrby(key, str(item_id), quantity)
```



```
def get_cart(user_id):
```

```
key = f'cart:{user_id*}'
```

```
return r.hgetall(key)
```


===== List 类型 =====

应用场景：消息队列、最新列表、时间线

右侧添加元素（队尾）

```
r.rpush('queue:tasks', 'task1', 'task2', 'task3')
```


左侧弹出元素（队头）


```
task = r.lpop('queue:tasks')          # 'task1'
```


范围获取（不删除）

```
tasks = r.lrange('queue:tasks', 0, -1)  # ['task2', 'task3']
```


固定长度列表（保留最新的N条）

```
r.lpush('recent_logins:user1001', '2024-01-15 09:00')
```

```
r.lpush('recent_logins:user1001', '2024-01-16 08:55')
```



```
ltrim('recent_logins:user1001', 0, 9)      # 只保留最近10条
```


阻塞式弹出（消息队列模式）

```
_, task = r.blpop('queue:tasks', timeout=30) # 最多等待30秒
```


===== Set 类型 =====

应用场景：标签、好友关系、去重、集合运算

添加成员

```
sadd('tags:article:100', 'AI', 'Python', 'MachineLearning')
```


成员判断

```
ismember('tags:article:100', 'AI')      # True
```


所有成员


```
smembers('tags:article:100')
```

```
# {'AI', 'Python', 'MachineLearning'}
```


集合运算

```
sadd('tags:article:101', 'Python', 'Web', 'Django')
```

```
sinter(['tags:article:100', 'tags:article:101']) # {'Python'* - 交
```



```
union(['tags:article:100', 'tags:article:101']) # 并集
```

```
sdiff(['tags:article:100', 'tags:article:101']) # 差集
```


社交应用：共同好友

```
def get_mutual_friends(user_a, user_b):
```

```
return sinter(f'friends:{user_a*}', f'friends:{user_b*}')
```


去重统计 (UV统计)

```
sadd('daily_visitors:20240115', 'ip1', 'ip2', 'ip3')
```

```
scard('daily_visitors:20240115')
```

```
# 3
```


===== Sorted Set 类型 =====

应用场景：排行榜、优先级队列、范围查询

添加成员（分数 + 成员）

```
zadd('leaderboard:game', {
```

```
'player1': 1500,
```



```
'player2': 1200,
```

```
'player3': 1800,
```

'player4': 1400

获取排行榜（按分数降序）

```
top_players = zrevrange('leaderboard:game', 0, 4, withscores=True)
```

```
# [('player3', 1800), ('player1', 1500), ('player4', 1400), ('player2', 1200)]
```


获取排名

```
rank = zrevrank('leaderboard:game', 'player1') # 1 (从0开始)
```


范围查询（分数区间）

```
top_scorers = zrangebyscore('leaderboard:game', 1400, 2000, withscores=True)
```


增加分数


```
zincrby('leaderboard:game', 100, 'player1') # player1变为1600
```


带权重的延迟任务队列

score = 时间戳, member = 任务ID

```
import time
```

```
task_time = int(time.time()) + 300 # 5分钟后执行
```

```
zadd('delayed_queue', {f'task_{task_time*}': task_time*})
```


获取到期任务

```
now = int(time.time())
```

```
due_tasks = zrangebyscore('delayed_queue', 0, now)
```

❑ 2.2 Python Redis客户端封装

```

# redis_client.py
import redis
from typing import Optional, Any, Dict, List
import json
import logging
from functools import wraps

logger = logging.getLogger(__name__)

class RedisCache:
    """Redis缓存封装类"""

    def __init__(self, host='localhost', port=6379, db=0, password=None):
        self.client = redis.Redis(
            host=host,
            port=port,
            db=db,
            password=password,
            decode_responses=True,
            socket_timeout=5,
            socket_connect_timeout=5,
            retry_on_timeout=True
        )

        # 测试连接
        try:
            self.client.ping()
            logger.info("Redis连接成功")
        except redis.ConnectionError:
            logger.error("Redis连接失败")
            raise

    def get(self, key: str) -> Optional[Any]:
        """获取缓存值"""
        try:
            value = self.client.get(key)
            if value is not None:
                return json.loads(value)
            return None
        except Exception as e:
            logger.error(f"Redis GET 失败 [{key*}]: {e*}")
            return None

    def set(self, key: str, value: Any, ttl: int = 3600) -> bool:
        """设置缓存值"""
        try:
            serialized = json.dumps(value, ensure_ascii=False)
            return self.client.setex(key, ttl, serialized)

```

```
except Exception as e:
```

```
logger.error(f"Redis SET 失败 [{key*}: {e*}")
```

```
return False
```



```
def delete(self, *keys: str) -> int:
```

""删除缓存""

try:

```
return self.client.delete(*keys)
```

```
except Exception as e:
```

```
logger.error(f"Redis DELETE 失败: {e*}")
```

```
return 0
```



```
def exists(self, *keys: str) -> bool:
```

""检查键是否存在""

```
return self.client.exists(*keys) == len(keys)
```



```
def increment(self, key: str, amount: int = 1) -> int:
```

""原子递增""

```
return self.client.incrby(key, amount)
```


===== 缓存装饰器 =====


```
def cache_decorator(self, ttl: int = 3600, key_prefix: str = ""):
```


缓存装饰器

用法：


```
@cache.cache_decorator(ttl=600, key_prefix="user:")
```

```
def get_user(user_id):
```



```
def decorator(func):
```

@wraps(func)

```
def wrapper(*args, **kwargs):
```

生成缓存键


```
cache_key = f"{key_prefix*{func.__name__}*:{str(args)*:{str(kwargs)*"
```


尝试从缓存获取

```
result = self.get(cache_key)
```

```
if result is not None:
```

```
logger.debug(f"□缓存命中: {cache_key*}")
```

return result

执行函数并缓存结果

```
result = func(*args, **kwargs)
```

```
self.set(cache_key, result, ttl)
```



```
return result
```

return wrapper

return decorator

全局实例

```
cache = RedisCache()
```

❏ 3. 缓存策略与问题解决（1小时）

❏ 3.1 常见缓存模式

Cache Aside Pattern（旁路缓存）：

```
# 读取流程
def get_user_with_cache(user_id: int):
    cache_key = f"user:{user_id}"

    # 1. 先查缓存
    user_data = cache.get(cache_key)
    if user_data:
        print("❏从缓存获取")
        return user_data

    # 2. 缓存未命中，查数据库
    print("❏从数据库获取")
    user_data = db.query(User).filter(User.id == user_id).first()

    if user_data:
        # 3. 写入缓存
        cache.set(cache_key, user_data.to_dict(), ttl=1800) # 30分
        钟

    return user_data

# 更新流程
def update_user(user_id: int, data: dict):
    cache_key = f"user:{user_id}"

    # 1. 先更新数据库
    db.query(User).filter(User.id == user_id).update(data)
    db.commit()

    # 2. 再删除缓存（不是更新！）
    cache.delete(cache_key)

    # 下次读取时会重新加载最新数据
```

Read-Through / Write-Through:

```

class CacheRepository:
    """
    Read-Through: 缓存未命中时自动加载
    Write-Through: 同步写入缓存和数据库
    """

    def __init__(self, db_session, cache_client):
        self.db = db_session
        self.cache = cache_client

    def get(self, key: str, loader_func=None, ttl: int = 3600):
        """Read-Through模式"""
        value = self.cache.get(key)

        if value is None and loader_func:
            value = loader_func()
            if value is not None:
                self.cache.set(key, value, ttl)

        return value

    def set(self, key: str, value: Any, ttl: int = 3600):
        """Write-Through模式"""
        # 同时写入缓存和数据库
        self.cache.set(key, value, ttl)
        # 这里应该调用db的写入方法
        # self.db.save(key, value)

    def delete(self, key: str):
        """同时删除缓存和数据"""
        self.cache.delete(key)
        # self.db.delete(key)

```

□ 3.2 缓存问题及解决方案

缓存穿透 (Cache Penetration) :

```

# 问题：查询不存在的数据，每次都穿透到数据库
# 解决方案1：布隆过滤器
from pybloom_live import ScalableBloomFilter

# 初始化布隆过滤器（加载所有有效ID）
bloom = ScalableBloomFilter(initial_capacity=10000, error_rate=0.001)

valid_ids = db.query(User.id).all()
for user_id in valid_ids:
    bloom.add(user_id)

def get_user_safe(user_id: int):
    # 快速判断ID是否可能存在
    if user_id not in bloom:
        return None # 一定不存在，直接返回

    # 可能存在，正常查询缓存和DB
    return get_user_with_cache(user_id)

# 解决方案2：缓存空值
def get_user_with_null_cache(user_id: int):
    cache_key = f"user:{user_id}"

    cached = cache.get(cache_key)
    if cached is not None:
        # 即使是None也返回（防止穿透）
        return cached.get('data') if isinstance(cached, dict) else cached

    user = db.query(User).filter(User.id == user_id).first()

    if user:
        cache.set(cache_key, user.to_dict(), ttl=1800)
    else:
        # 缓存空结果，但TTL较短
        cache.set(cache_key, {'data': None}, ttl=60) # 只缓存1分钟

    return user

```

缓存击穿（Cache Breakdown）：

```
# 问题：热点Key过期瞬间，大量请求同时打到数据库
# 解决方案：互斥锁（Mutex Lock）

import threading

locks = {}

def get_hot_data_with_lock(key: str):
    # 1. 查缓存
    data = cache.get(key)
    if data is not None:
        return data

    # 2. 缓存未命中，获取互斥锁
    lock_key = f"lock:{key}"

    # 尝试获取分布式锁
    if cache.setnx(lock_key, "locked", 10): # 锁10秒自动释放
        try:
            # 获得锁，查询数据库
            print(f"□获取锁成功，查询数据库: {key*}")
            data = load_from_database(key)

            # 写入缓存（设置随机过期时间，防止雪崩）
            import random
            ttl = 1800 + random.randint(0, 300) # 30-35分钟
            cache.set(key, data, ttl)

            return data
        finally:
            cache.delete(lock_key)
    else:
        # 未获得锁，短暂等待后重试
        import time
        time.sleep(0.05) # 50ms
        return get_hot_data_with_lock(key) # 递归重试
```

缓存雪崩（Cache Avalanche）：

```

# 问题：大量Key同时过期，或Redis宕机
# 解决方案1: TTL加随机值
import random

def set_cache_with_random_ttl(key: str, value: Any, base_ttl: int = 1800):
    """设置缓存时添加随机偏移量"""
    jitter = random.randint(0, base_ttl // 10) # *10%的随机偏移
    actual_ttl = base_ttl + jitter
    cache.set(key, value, actual_ttl)

# 解决方案2: 多级缓存（本地缓存 + Redis）
from cachetools import TTLCache

local_cache = TTLCache(maxsize=1000, ttl=60) # 本地缓存1分钟

def get_with_multi_level_cache(key: str):
    # Level 1: 本地缓存（最快）
    data = local_cache.get(key)
    if data is not None:
        return data

    # Level 2: Redis缓存
    data = cache.get(key)
    if data is not None:
        local_cache[key] = data # 回填本地缓存
        return data

    # Level 3: 数据库
    data = load_from_database(key)
    if data:
        cache.set(key, data, 1800)
        local_cache[key] = data

    return data

# 解决方案3: 熔断降级
class CircuitBreaker:
    """熔断器"""

    def __init__(self, failure_threshold=5, recovery_timeout=60):
        self.failure_count = 0
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.last_failure_time = None
        self.state = 'closed' # closed, open, half-open

    def call(self, func, *args, **kwargs):

```



```
if self.state == 'open':
```

```
if time.time() - self.last_failure_time > self.recovery_timeout:
```

```
self.state = 'half-open'
```

else:

```
raise Exception("Circuit breaker is OPEN - service unavailable")
```


try:

```
result = func(*args, **kwargs)
```



```
self.on_success()
```

return result

```
except Exception as e:
```

```
self.on_failure()
```

raise e


```
def on_success(self):
```

```
self.failure_count = 0
```



```
self.state = 'closed'
```



```
def on_failure(self):
```

```
self.failure_count += 1
```

```
self.last_failure_time = time.time()
```

```
if self.failure_count >= self.failure_threshold:
```

```
self.state = 'open'
```

□4. 会话管理与认证 (1小时)

□4.1 Redis Session 存储


```

# session_manager.py
import uuid
import json
import time
from typing import Optional, Dict, Any
from redis import Redis

class RedisSessionManager:
    """基于Redis的Session管理器"""

    def __init__(self, redis_client: Redis, prefix: str = "session:", ttl: int = 86400):
        self.redis = redis_client
        self.prefix = prefix
        self.ttl = ttl # 默认24小时过期

    def create_session(self, user_data: Dict[str, Any]) -> str:
        """创建新会话"""
        session_id = str(uuid.uuid4()).replace('-', '')
        session_key = f"{self.prefix}{session_id}"

        session_data = {
            'session_id': session_id,
            'user_id': user_data['id'],
            'username': user_data['username'],
            'role': user_data.get('role', 'user'),
            'created_at': time.time(),
            'last_accessed': time.time(),
            'ip_address': user_data.get('ip'),
            'user_agent': user_data.get('user_agent')
        }

        # 存储到Redis
        self.redis.setex(
            session_key,
            self.ttl,
            json.dumps(session_data, ensure_ascii=False)
        )

        # 记录用户的活跃会话
        active_sessions_key = f"{self.prefix}user:{user_data['id']}:sessions"
        self.redis.sadd(active_sessions_key, session_id)

        return session_id

    def get_session(self, session_id: str) -> Optional[Dict]:
        """获取会话信息"""
        session_key = f"{self.prefix}{session_id}"
        data = self.redis.get(session_key)

```



```
if data is None:
```

```
return None
```



```
session_data = json.loads(data)
```


更新最后访问时间（懒更新，降低写压力）


```
if random.random() < 0.2: # 20%概率更新
```

```
session_data['last_accessed'] = time.time()
```

```
self.redis.setex(session_key, self.ttl, json.dumps(session_data))
```



```
return session_data
```



```
def update_session(self, session_id: str, update_data: Dict) -> bool:
```

""更新会话数据""


```
session_key = f"{self.prefix*{session_id}"
```

```
existing = self.get_session(session_id)
```



```
if existing is None:
```

```
return False
```



```
existing.update(update_data)
```

```
existing['last_accessed'] = time.time()
```



```
self.redis.setex(session_key, self.ttl, json.dumps(existing))
```

```
return True
```



```
def destroy_session(self, session_id: str) -> bool:
```

""销毁会话（登出）""

```
session_key = f"{self.prefix*{session_id}"
```

```
session_data = self.get_session(session_id)
```



```
if session_data:
```

从用户活跃会话集合中移除

```
active_sessions_key = f"{self.prefix*user:{session_data['user_id']*:sessio
```

ns"

```
self.redis.srem(active_sessions_key, session_id)
```



```
return self.redis.delete(session_key) > 0
```



```
def destroy_all_user_sessions(self, user_id: int) -> int:
```

"""销毁用户的所有会话（强制重新登录）"""

```
active_sessions_key = f"{self.prefix*user:{user_id*:sessions}"
```

```
session_ids = self.redis.smembers(active_sessions_key)
```



```
count = 0
```

```
for sid in session_ids:
```



```
session_key = f"{self.prefix*{sid}"
```

```
count += self.redis.delete(session_key)
```


清空活跃会话集合

```
self.redis.delete(active_sessions_key)
```


return count


```
def list_active_sessions(self, user_id: int) -> list:
```

"""列出用户的所有活跃会话"""

```
active_sessions_key = f"{self.prefix*user:{user_id*:sessions}"
```

```
session_ids = self.redis.smembers(active_sessions_key)
```



```
sessions = []
```

```
for sid in session_ids:
```

```
session_data = self.get_session(sid)
```



```
if session_data:
```

```
sessions.append(session_data)
```


return sessions

4.2 FastAPI集成Redis Session

```

# dependencies.py
from fastapi import Request, Depends, HTTPException, Cookie
from typing import Optional
from session_manager import RedisSessionManager
import redis

redis_client = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)
session_manager = RedisSessionManager(redis_client)

async def get_current_user(
    request: Request,
    session_id: Optional[str] = Cookie(None)
):
    """从Cookie获取当前登录用户"""
    if not session_id:
        raise HTTPException(status_code=401, detail="未登录")

    session = session_manager.get_session(session_id)

    if session is None:
        raise HTTPException(status_code=401, detail="会话已过期，请重新登录")

    # 检查IP是否变化（可选的安全措施）
    client_ip = request.client.host
    if session.get('ip_address') and session['ip_address'] != client_ip:
        # IP不一致，可能是会话劫持
        session_manager.destroy_session(session_id)
        raise HTTPException(status_code=401, detail="安全检测失败，请重新登录")
)

    return session

# routers/auth.py
from fastapi import APIRouter, Response
from pydantic import BaseModel

router = APIRouter(prefix="/auth", tags=["Authentication"])

class LoginRequest(BaseModel):
    username: str
    password: str

@router.post("/login")
async def login(credentials: LoginRequest, response: Response):
    """用户登录"""
    # 验证凭据
    user = authenticate_user(credentials.username, credentials.password)

```

```
if not user:
```

```
raise HTTPException(status_code=401, detail="用户名或密码错误")
```


创建会话

```
request_data = {
```

```
'id': user.id,
```

```
'username': user.username,
```

```
'role': user.role,
```

```
'ip': '127.0.0.1', # 从request中获取真实IP
```

'user_agent': 'Mozilla/5.0...'


```
session_id = session_manager.create_session(request_data)
```


设置Cookie

```
response.set_cookie(
```

```
key="session_id",
```

value=session_id,

httponly=True, # 防止XSS攻击

secure=True, # 仅HTTPS传输

```
samesite='lax',      # CSRF保护
```

max_age=86400

24小时


```
return {"message": "登录成功", "user": user.to_dict()*
```



```
@router.post("/logout")
```

```
async def logout(response: Response, current_user = Depends(get_current_user)):
```

""用户登出""

```
session_manager.destroy_session(current_user['session_id'])
```



```
response.delete_cookie("session_id")
```

```
return {"message": "已退出登录"}
```



```
@router.get("/me")
```

```
async def get_me(current_user = Depends(get_current_user)):
```

"""获取当前用户信息"""

```
return current_user
```



```
@router.post("/logout-all")
```

```
async def logout_all_devices(current_user = Depends(get_current_user)):
```

""注销所有设备的登录状态""

```
count = session_manager.destroy_all_user_sessions(current_user['user_id'])
```

```
return {"message": f"已注销{count}个设备的登录状态"}
```

*> 实践任务清单

☐任务1：设计博客系统的数据库（1.5小时）

需求分析：

博客系统需要支持：

- ☐ 用户管理（注册/登录/权限）
- ☐ 文章发布（草稿/发布/归档）
- ☐ 评论互动（嵌套评论）
- ☐ 分类标签（多对多关系）
- ☐ 点赞收藏（计数功能）

ER图设计：

Users (用户)

- +-- id (PK)
- +-- username (UNIQUE)
- +-- email (UNIQUE)
- +-- password_hash
- +-- avatar_url
- +-- bio
- +-- role (admin/editor/author/subscriber)
- +-- timestamps

Posts (文章)

- +-- id (FK -> Users.id)
- +-- title
- +-- slug (URL友好标识, UNIQUE)
- +-- content (TEXT, 支持Markdown)
- +-- excerpt (摘要)
- +-- cover_image
- +-- status (draft/published/archived)
- +-- view_count
- +-- like_count
- +-- published_at
- +-- timestamps

Categories (分类)

- +-- id
- +-- name (UNIQUE)
- +-- slug
- +-- description
- +-- parent_id (自引用, FK -> Categories.id)

Tags (标签)

- +-- id
- +-- name (UNIQUE)
- +-- slug

Comments (评论)

- +-- id
- +-- post_id (FK -> Posts.id)
- +-- user_id (FK -> Users.id)
- +-- parent_id (FK -> Comments.id, 自引用用于嵌套)
- +-- content
- +-- status (pending/approved/spam)
- +-- timestamps

Post_Category (文章-分类, 多对多)

- +-- post_id (FK)
- +-- category_id (FK)

Post_Tag (文章-标签, 多对多)

+-- post_id (FK)

+-- tag_id (FK)

Likes (点赞)

```
+-- user_id (FK -> Users.id)
```

```
+-- target_type ('post'/'comment')
```

`+-- target_id`


```
+-- user_id (FK -> Users.id)
```

```
+-- post_id (FK -> Posts.id)
```

SQL实现:

-- 创建扩展

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
```

-- 用户表

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    display_name VARCHAR(100),
    avatar_url VARCHAR(500),
    bio TEXT,
    role VARCHAR(20) NOT NULL DEFAULT 'subscriber'
        CHECK (role IN ('admin', 'editor', 'author', 'subscriber')),
    is_active BOOLEAN DEFAULT TRUE,
    last_login_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_username ON users(username);
```

-- 文章表

```
CREATE TABLE posts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    author_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    slug VARCHAR(250) UNIQUE NOT NULL,
    content TEXT NOT NULL,
    excerpt TEXT,
    cover_image VARCHAR(500),
    status VARCHAR(20) NOT NULL DEFAULT 'draft'
        CHECK (status IN ('draft', 'published', 'archived')),
    view_count INTEGER DEFAULT 0,
    like_count INTEGER DEFAULT 0,
    comment_count INTEGER DEFAULT 0,
    published_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_posts_author ON posts(author_id);
CREATE INDEX idx_posts_status_published ON posts(status, published_at DESC)
    WHERE status = 'published';
CREATE INDEX idx_posts_slug ON posts(slug);
CREATE INDEX idx_posts_title_trgm ON posts USING gin(title gin_trgm_ops);
```

— 分类表（支持层级）

CREATE TABLE categories (

```
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

name VARCHAR(100) UNIQUE NOT NULL,

slug VARCHAR(120) UNIQUE NOT NULL,

description TEXT,

```
parent_id UUID REFERENCES categories(id) ON DELETE SET NULL,
```

```
sort_order INTEGER DEFAULT 0,
```

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);


```
CREATE INDEX idx_categories_parent ON categories(parent_id);
```


一 标签表

CREATE TABLE tags (

```
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

name VARCHAR(50) UNIQUE NOT NULL,

slug VARCHAR(60) UNIQUE NOT NULL,

```
usage_count INTEGER DEFAULT 0,
```

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);

— 多对多关联表

```
CREATE TABLE post_categories (
```

```
post_id UUID REFERENCES posts(id) ON DELETE CASCADE,
```

```
category_id UUID REFERENCES categories(id) ON DELETE CASCADE,
```

PRIMARY KEY (post_id, category_id)

);


```
CREATE TABLE post_tags (
```

```
post_id UUID REFERENCES posts(id) ON DELETE CASCADE,
```

```
tag_id UUID REFERENCES tags(id) ON DELETE CASCADE,
```

PRIMARY KEY (post_id, tag_id)

);

一 评论表（支持嵌套回复）

CREATE TABLE comments (

```
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
```

```
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
```

```
parent_id UUID REFERENCES comments(id) ON DELETE CASCADE,
```

content TEXT NOT NULL,

status VARCHAR(20) DEFAULT 'pending'

```
CHECK (status IN ('pending', 'approved', 'spam', 'deleted')),
```



```
like_count INTEGER DEFAULT 0,
```

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
```

updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);


```
CREATE INDEX idx_comments_post ON comments(post_id, created_at DESC);
```

```
CREATE INDEX idx_comments_parent ON comments(parent_id);
```

```
CREATE INDEX idx_comments_user ON comments(user_id);
```


一 点赞表

```
CREATE TABLE likes (
```

```
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
```

```
target_type VARCHAR(20) NOT NULL CHECK (target_type IN ('post', 'comment')),
```

target_id UUID NOT NULL,

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
```

PRIMARY KEY (user_id, target_type, target_id)

);

一 收藏表

CREATE TABLE bookmarks (

```
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
```

```
post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
```

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
```

PRIMARY KEY (user_id, post_id)

);

-- 触发器：自动更新updated_at

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
```

RETURNS TRIGGER AS \$\$

BEGIN

```
NEW.updated_at = NOW();
```

RETURN NEW;

END;

```
$$ language 'plpgsql';
```



```
CREATE TRIGGER update_users_updated_at BEFORE UPDATE ON users
```

```
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```



```
CREATE TRIGGER update_posts_updated_at BEFORE UPDATE ON posts
```

```
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```


检验标准：

- ☐ [] 所有表创建成功且符合规范
 - ☐ [] 外键约束正确建立
 - ☐ [] 索引覆盖主要查询场景
 - ☐ [] 支持嵌套评论和层级分类
 - ☐ [] 能处理并发点赞/收藏（通过唯一约束）
-

☐任务2：实现缓存层（1.5小时）

任务要求：

为博客API添加完整的缓存层：

- ☐ 首页缓存
- ☐ 热门文章列表（TTL：10分钟）
- ☐ 最新文章列表（TTL：5分钟）
- ☐ 推荐分类和标签（TTL：1小时）
- ☐ 文章详情缓存
- ☐ 文章完整内容（TTL：30分钟）
- ☐ 文章评论列表（TTL：10分钟）
- ☐ 相关文章推荐（TTL：1小时）
- ☐ 用户相关缓存
- ☐ 用户基本信息（TTL：1小时）
- ☐ 用户发布的文章数（TTL：30分钟）
- ☐ 用户关注列表（TTL：10分钟）
- ☐ 计数器缓存
- ☐ 文章浏览量（异步批量更新）
- ☐ 点赞数（先写Redis再同步DB）
- ☐ 评论数（实时更新）

代码框架：

```

# services/cache_service.py
from typing import List, Optional, Any
from redis import Redis
import json
import logging

logger = logging.getLogger(__name__)

class BlogCacheService:
    """博客系统缓存服务"""

    CACHE_PREFIXES = {
        'post_detail': 'blog:post:',
        'post_list': 'blog:posts:',
        'user_info': 'blog:user:',
        'category': 'blog:category:',
        'tag': 'blog:tag:',
        'hot_posts': 'blog:hot:',
        'counter': 'blog:count:'
    }
    *

    def __init__(self, redis_client: Redis):
        self.redis = redis_client

    # ===== 文章缓存 =====

    def get_post(self, post_id: str) -> Optional[dict]:
        """获取文章详情"""
        key = f"{self.CACHE_PREFIXES['post_detail']}{post_id}"
        data = self.redis.get(key)
        return json.loads(data) if data else None

    def set_post(self, post_id: str, post_data: dict, ttl: int = 1800):
        """设置文章缓存"""
        key = f"{self.CACHE_PREFIXES['post_detail']}{post_id}"
        self.redis.setex(key, ttl, json.dumps(post_data, ensure_ascii=False, default=s
tr))

    def delete_post(self, post_id: str):
        """删除文章缓存（文章更新时调用）"""
        key = f"{self.CACHE_PREFIXES['post_detail']}{post_id}"
        self.redis.delete(key)
        # 同时清除相关的列表缓存
        self.clear_post_list_caches()

    # ===== 文章列表缓存 =====

    def get_post_list(self, page: int, category: str = None, tag: str = None) -> Optio
nal[list]:

```

""获取缓存的文章列表""

```
cache_key_parts = ['list', page]
```

```
if category:
```

```
cache_key_parts.append(f'cat:{category*}')
```

```
if tag:
```

```
cache_key_parts.append(f'tag:{tag*}')
```



```
key = f"{self.CACHE_PREFIXES['post_list']*{'.'.join(cache_key_parts)*"
```

```
data = self.redis.get(key)
```

```
return json.loads(data) if data else None
```



```
def set_post_list(self, page: int, posts: list, category: str = None, tag: str = N
```

```
one, ttl: int = 600):
```

"""设置文章列表缓存"""


```
cache_key_parts = ['list', page]
```

```
if category:
```

```
cache_key_parts.append(f'cat:{category*}')
```

```
if tag:
```

```
cache_key_parts.append(f'tag:{tag*}')
```



```
key = f"{self.CACHE_PREFIXES['post_list']*{'.'.join(cache_key_parts)*"
```

```
self.redis.setex(key, ttl, json.dumps(posts, ensure_ascii=False, default=str))
```



```
def clear_post_list_caches(self):
```

"""清除所有文章列表缓存（新文章发布时调用）"""

```
pattern = f"{self.CACHE_PREFIXES['post_list']}:*"
```

```
keys = self.redis.keys(pattern)
```

```
if keys:
```

```
self.redis.delete(*keys)
```


===== 计数器 =====


```
def increment_view_count(self, post_id: str):
```

"""增加浏览量（写入Redis，异步同步到DB）"""

```
key = f"{self.CACHE_PREFIXES['counter']*views:{post_id}"
```

```
self.redis.incr(key)
```


如果是第一次访问，设置过期时间


```
if self.redis.ttl(key) == -1:
```

```
self.redis.expire(key, 3600) # 1小时后过期，触发批量同步
```



```
def get_view_count(self, post_id: str) -> int:
```

"""获取浏览量（优先读Redis）"""

```
key = f"{self.CACHE_PREFIXES['counter']*views:{post_id}"
```

```
count = self.redis.get(key)
```

```
return int(count) if count else 0
```



```
def increment_like_count(self, post_id: str, user_id: str, liked: bool):
```

""更新点赞数""

if liked:

使用Sorted Set记录点赞用户，同时作为计数器

```
pipe = self.redis.pipeline()
```

```
pipe.zadd(f"{self.CACHE_PREFIXES['counter']*likes:{post_id*}", {user_id: ti
```

```
me.time(*)
```



```
pipe.expire(f"{self.CACHE_PREFIXES['counter']*likes:{post_id}", 86400)
```

```
pipe.execute()
```

else:

```
self.redis.zrem(f"{self.CACHE_PREFIXES['counter']*likes:{post_id}", user_i
```



```
def get_like_count(self, post_id: str) -> int:
```

""获取点赞数""


```
key = f"{self.CACHE_PREFIXES['counter']*likes:{post_id}"
```

```
return self.redis.zcard(key)
```



```
def check_user_liked(self, post_id: str, user_id: str) -> bool:
```

""检查用户是否已点赞""

```
key = f"{self.CACHE_PREFIXES['counter']*likes:{post_id}"
```

```
return self.redis.zscore(key, user_id) is not None
```


===== 热门内容排行 =====


```
def add_to_hot_posts(self, post_id: str, score: float):
```

""添加到热门文章排行榜（基于浏览量和时间）""

```
self.redis.zadd('blog:hot_posts:all', {post_id: score*})
```

只保留前100篇热门文章

```
self.redis.zremrangebyrank('blog:hot_posts:all', 0, -101)
```



```
def get_hot_posts(self, limit: int = 10) -> list:
```

"""获取热门文章ID列表"""

```
return self.redis.zrevrange('blog:hot_posts:all', 0, limit - 1)
```


===== 批量同步（定时任务调用） =====


```
def sync_view_counts_to_db(self, db_session):
```

"""将Redis中的浏览量批量同步到数据库"""


```
pattern = f"{self.CACHE_PREFIXES['counter']*views:"
```

```
keys = self.redis.keys(pattern)
```


for key in keys:

```
post_id = key.split(':')[1]
```

```
count = int(self.redis.get(key) or 0)
```



```
if count > 0:
```


更新数据库

```
db_session.execute(
```

```
"UPDATE posts SET view_count = view_count + :count WHERE id = :pos
```

t_id",

```
{'count': count, 'post_id': post_id*
```


清除Redis计数器


```
self.redis.delete(key)
```



```
db_session.commit()
```

```
logger.info(f"已同步 {len(keys)* 个文章的浏览量")
```


使用示例

在FastAPI路由中使用


```
@router.get("/posts/{post_id*}")
```

```
async def get_post(post_id: str, db: Session = Depends(get_db)):
```

先查缓存

```
post = blog_cache.get_post(post_id)
```



```
if post:
```

异步增加浏览量

```
await asyncio.to_thread(blog_cache.increment_view_count, post_id)
```


return post

缓存未命中，查数据库

```
post = db.query(Post).filter(Post.id == post_id).first()
```

if not post:

```
raise HTTPException(404, "文章不存在")
```


写入缓存


```
blog_cache.set_post(post_id, post.to_dict())
```


增加浏览量

```
await asyncio.to_thread(blog_cache.increment_view_count, post_id)
```


return post

检验标准：

- ☐ [] 首页加载速度提升（对比缓存前后响应时间）
- ☐ [] 文章详情页能正确缓存和失效
- ☐ [] 浏览量计数准确（Redis和DB最终一致）
- ☐ [] 点赞功能正常工作
- ☐ [] 热门文章排行榜实时更新
- ☐ [] 缓存穿透/击穿有防护措施

** 推荐学习资源

☐ PostgreSQL资源

- ☐ PostgreSQL官方文档 - 最权威的参考
- ☐ 高性能PostgreSQL - 性能优化指南
- ☐ Explain Visualizer - 执行计划可视化工具

☐ Redis资源

- ☐ Redis官方文档 - 命令和概念详解
- ☐ Redis最佳实践 - 生产环境经验
- ☐ Redis.io - 商业版和企业方案

☐ 工具推荐

- ☐ pgAdmin / DBeaver - PostgreSQL GUI客户端
- ☐ RedisInsight - Redis可视化工具
- ☐ DataGrip - JetBrains的多数据库IDE
- ☐ Prometheus + Grafana - 数据库监控方案

✓ 今日自测题（精简版）

☐ 关键知识点检测

- ☐ PostgreSQL中，什么时候应该使用反规范化？
 - ☐ A. 总是应该使用
 - ☐ B. 读远多于写的场景，为了减少JOIN
 - ☐ C. 写入频繁的场景
 - ☐ D. 需要强一致性的场景
- ☐ 以下哪个Redis数据类型最适合实现排行榜？

- ☐ A. String
- ☐ B. Hash
- ☐ C. Sorted Set
- ☐ D. List

- ☐ 如何解决缓存穿透问题？（选择所有适用）
- ☐ A. 布隆过滤器
- ☐ B. 缓存空值
- ☐ C. 互斥锁
- ☐ D. 设置随机TTL

- ☐ Redis的SETNX命令适用于什么场景？
- ☐ A. 批量设置键值
- ☐ B. 实现分布式锁
- ☐ C. 计数器递增
- ☐ D. 列表操作

- ☐ PostgreSQL的CTE（公用表表达式）的主要优势？
- ☐ A. 提高查询性能
- ☐ B. 提高代码可读性和复用性
- ☐ C. 减少内存使用
- ☐ D. 自动创建索引

☐答案

- ☐ B - 反规范化以牺牲空间换时间，适合读多写少场景。
- ☐ C - Sorted Set天然支持排序和排名。
- ☐ AB - C是解决击穿，D是解决雪崩。
- ☐ B - SETNX = SET if Not Exists，原子性操作适合锁。
- ☐ B - CTE让复杂SQL更清晰易维护。

** 今日总结

☐关键收获

- ☐ _____
- ☐ _____
- ☐ _____

☐明日预告

Day 5: 运维与部署实践 将涵盖：

- ☐ Docker Compose多容器编排
- ☐ CI/CD流水线搭建
- ☐ Nginx反向代理配置
- ☐ 监控和日志收集
- ☐ 生产环境部署最佳实践

准备工作:

- ☐ [] 确保Day2-4的项目可以正常运行
- ☐ [] 注册Docker Hub账号 (可选)
- ☐ [] 准备云服务器 (阿里云/腾讯云/AWS等)

** 实战项目：高性能缓存系统 ☐

☐项目概览

项目名称: Cache Demo - Redis缓存策略演示 路径:04-数据库技术应用/cache-demo/ 完成度: ✓
100% 文件数: 6个核心文件 代码量: 800+ 行 技术栈: Python + Redis + PostgreSQL + FastAPI

☐核心特性

✓ Redis完整封装 - 基础CRUD + 连接池管理 + 统计监控 ✓ 三大缓存防护机制 - 穿透/击穿/雪崩解决方案 ✓ PostgreSQL连接池 - 慢查询监控 + 性能优化 ✓ FastAPI中间件 - @cached() 装饰器 + 自动缓存失效 ✓ 性能测试套件 - 对比缓存前后性能提升

☐项目架构

```
cache-demo/
+-- redis_cache.py          # Redis客户端封装 (500+行)
|   +-- 基础操作: get/set/delete/exists/ttl
|   +-- 连接池管理
|   +-- 三大防护: null_cache/mutex_lock/random_ttl
|
+-- postgres_config.py      # PostgreSQL连接池配置
|   +-- 慢查询事件监听器
|
+-- caching_middleware.py   # FastAPI缓存中间件
|   +-- @cached() 函数级装饰器
|
+-- performance_test.py     # 性能测试脚本
    +-- 基础操作性能对比
    +-- 缓存穿透防护测试
    +-- 缓存击穿防护测试
    +-- 缓存雪崩防护测试
```

核心功能详解

1. Redis缓存客户端 (redis_cache.py)

```
# 三大缓存问题解决方案

# 防穿透：缓存空值
cache.get_with_null_cache(key, loader_func, ttl=60)

# 防击穿：互斥锁
cache.get_with_mutex_lock(key, loader_func, lock_timeout=10)

# 防雪崩：随机TTL + 多级缓存
cache.get_with_random_ttl(key, loader_func, base_ttl=1800)
```

2. FastAPI缓存装饰器 (caching_middleware.py)

```
# 使用示例
@cached(ttl=300, key_prefix="user:")
def get_user_profile(user_id: int):
    # 自动缓存结果300秒
    return db.query(User).get(user_id)
```

3. 性能测试结果（预期）

操作类型	无缓存	有缓存	提升倍数
基础GET	~50ms	~2ms	25x
批量MGET	~200ms	~10ms	20x
热点查询	~100ms	~3ms	33x

快速启动

```
# 1. 进入项目目录
cd 04-数据库技术应用/cache-demo

# 2. 启动Redis和PostgreSQL (Docker)
docker run -d --name redis -p 6379:6379 redis:7-alpine
docker run -d --name postgres -p 5432:5432 \
  -e POSTGRES_PASSWORD=test \
  postgres:15-alpine

# 3. 安装依赖
pip install -r requirements.txt

# 4. 运行性能测试
python performance_test.py

# 5. 查看测试报告
# 输出包含各项操作的耗时对比和命中率统计
```

□验收标准

- ☐ [] Redis基础操作正常 (get/set/delete)
- ☐ [] 连接池配置生效 (可查看连接数)
- ☐ [] 缓存穿透防护有效 (空值缓存命中)
- ☐ [] 缓存击穿防护有效 (互斥锁防止并发)
- ☐ [] 缓存雪崩防护有效 (TTL随机化)
- ☐ [] @cached() 装饰器正常工作
- ☐ [] 性能测试通过 (缓存命中率 > 90%)

*□模块导航

Day 3: 后端开发技能提升 | Day 5: 运维与部署实践 | ✓ □返回课程首页

√* Day 4 完成! 你已掌握双数据库的高效应用!

明日将学习DevOps技能, 让你的应用能够上线运行!